

Практическая работа 23. Загрузка и отображение изображений из Интернета

В предыдущих работах вы узнали, как получить данные из веб-сервиса с помощью **паттерна репозитория** и разобрать ответ на объект Kotlin. В этом уроке вы используете полученные знания для загрузки и отображения фотографий с веб-адреса. Кроме того, вы повторите, как создать **LazyVerticalGrid** и использовать его для отображения сетки изображений на странице обзора.

Необходимые условия

- Знания о том, как получать JSON из веб-сервиса REST и разбирать эти данные на объекты Kotlin с помощью библиотек Retrofit и Gson
- Знание веб-сервиса REST
- Знакомство с компонентами архитектуры Android, такими как слой данных и репозиторий
- Знание инъекции зависимостей
- Знание ViewModel и ViewModelProvider.Factory
- Знание реализации корутин для вашего приложения
- Знание паттерна репозитория

Что вы узнаете

- Как использовать библиотеку Coil для загрузки и отображения изображения с веб-адреса.
- Как использовать LazyVerticalGrid для отображения сетки изображений.
- Как обрабатывать возможные ошибки при загрузке и отображении изображений.

Что вы создадите

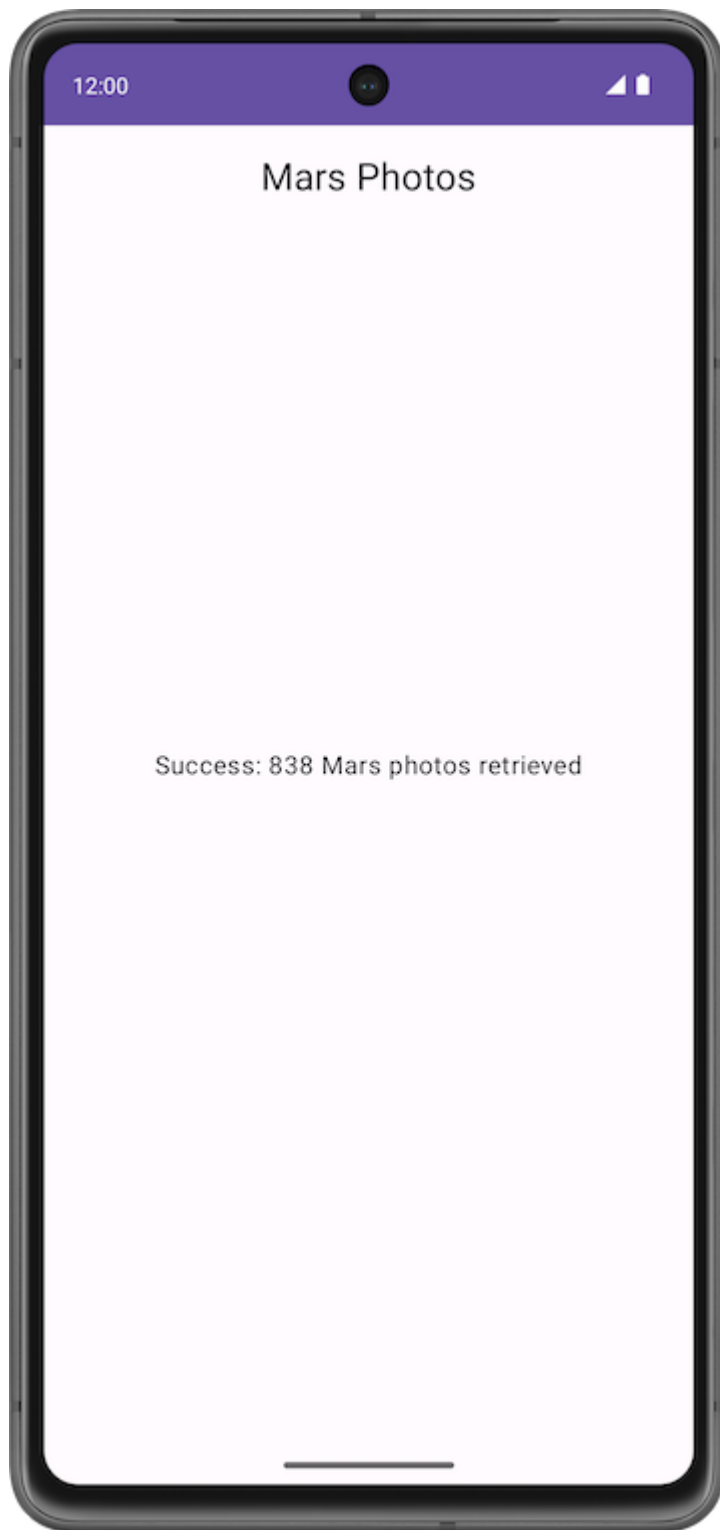
- Модифицируйте приложение Mars Photos, чтобы получить URL-адрес изображения из данных Марса и использовать Coil для загрузки и отображения этого изображения.
- Добавьте в приложение анимацию загрузки и значок ошибки.
- Добавьте в приложение обработку статусов и ошибок.

Что вам понадобится

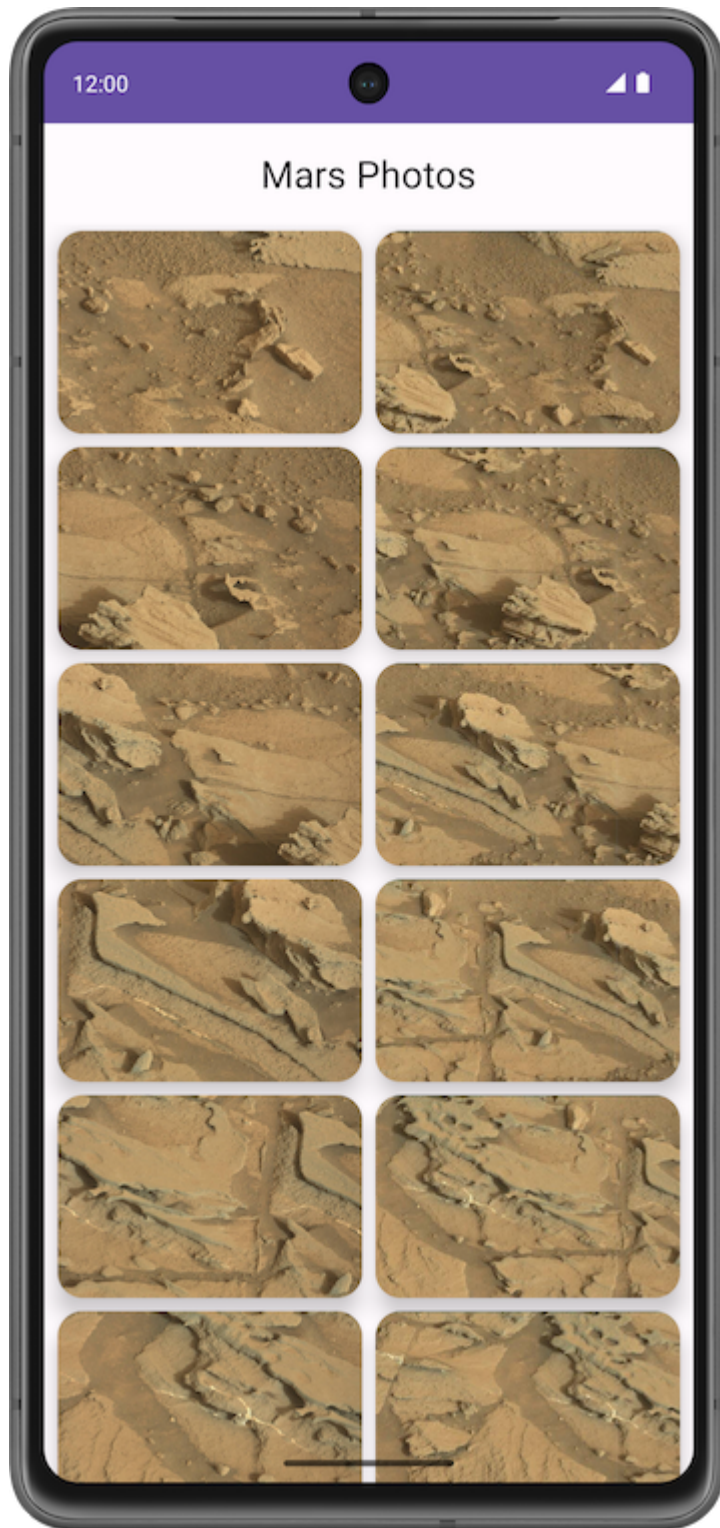
- Компьютер с современным веб-браузером, например последней версией Chrome.
- Стартовый код приложения Mars Photos с веб-сервисами REST

Обзор приложения

В этом мастер-классе вы продолжите работу с приложением Mars Photos из предыдущего мастер-класса. Приложение Mars Photos подключается к веб-сервису для получения и отображения количества объектов Kotlin, полученных с помощью Gson. Эти объекты Kotlin содержат URL-адреса реальных фотографий с поверхности Марса, сделанных марсоходами НАСА.



Версия приложения, которую вы создадите в этом codelab, отображает фотографии Марса в виде сетки изображений. Изображения являются частью данных, которые ваше приложение получает из веб-сервиса. Ваше приложение использует библиотеку Coil для загрузки и отображения изображений и LazyVerticalGrid для создания макета сетки для изображений. Ваше приложение также будет изящно обрабатывать сетевые ошибки, выводя сообщение об ошибке.



Получите стартовый код Чтобы начать работу, загрузите стартовый код:

```
$ git clone https://github.com/google-developer-training/basic-android-kotlin-compose-training-mars-photos.git
$ cd basic-android-kotlin-compose-training-mars-photos
$ git checkout coil-starter
```

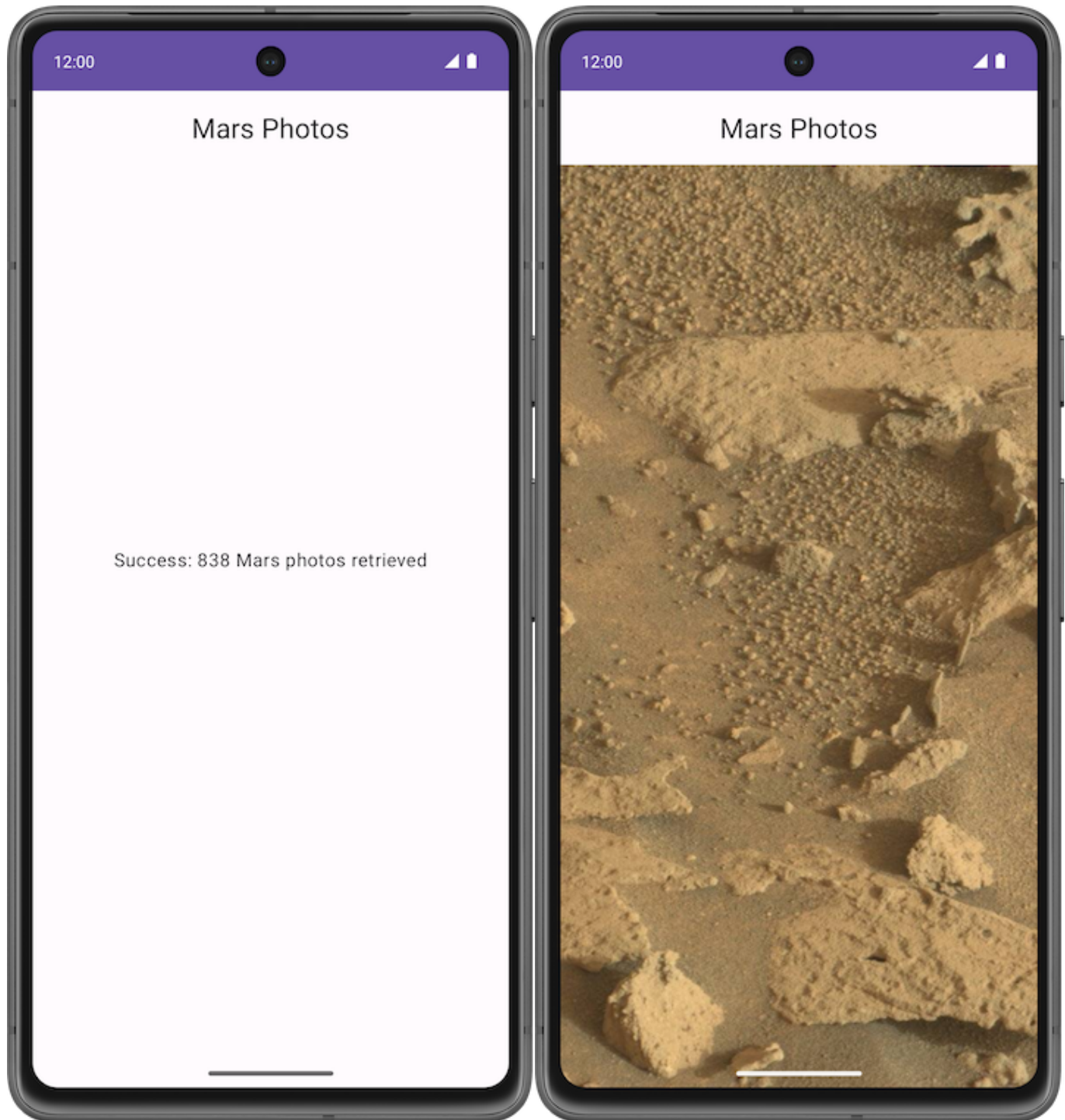
3. Отображение загруженного изображения

Отображение фотографии с веб-адреса может показаться простым, но для того, чтобы оно работало хорошо, необходимо проделать немало технических операций. Изображение должно быть загружено, внутренне сохранено (кэшировано) и декодировано из сжатого формата в изображение, которое может использовать Android. Вы можете кэшировать изображение в кэш в памяти, в кэш на базе хранилища или в оба. Все это должно происходить в низкоприоритетных фоновых потоках, чтобы пользовательский интерфейс оставался отзывчивым. Кроме того, для лучшей производительности сети и процессора вам может понадобиться получить и декодировать несколько изображений одновременно.

К счастью, вы можете использовать разработанную сообществом библиотеку под названием Coil для загрузки, буферизации, декодирования и кэширования изображений. Без использования Coil у вас было бы гораздо больше работы.

В основном Coil нужны две вещи:

URL-адрес изображения, которое вы хотите загрузить и отобразить. Композитный AsyncImage для фактического отображения этого изображения. В этом задании вы узнаете, как использовать Coil для отображения одного изображения из веб-сервиса Mars. Вы отображаете изображение первой фотографии Марса в списке фотографий, которые возвращает веб-сервис. На следующих изображениях показаны скриншоты до и после:



Добавьте зависимость Coil

Откройте приложение Mars Photos из репозитория Add и Manual DI codelab. Запустите приложение, чтобы убедиться, что оно показывает количество полученных фотографий Марса. Откройте `build.gradle.kts` (модуль `:app`). В разделе зависимостей добавьте эту строку для библиотеки Coil:

```
// Coil
implementation("io.coil-kt:coil-compose:2.4.0")
```

Проверьте и обновите последнюю версию библиотеки со страницы документации Coil.

Нажмите Sync Now, чтобы перестроить проект с новой зависимостью. Отображение URL-адреса изображения На этом шаге вы получите и отобразите URL первой фотографии Марса.

В файле `ui/screens/MarsViewModel.kt` в методе `getMarsPhotos()` в блоке `try` найдите строку, которая устанавливает данные, полученные из веб-сервиса, в `listResult`.

```
// No need to copy, code is already present
try {
    val listResult = marsPhotosRepository.getMarsPhotos()
    //...
}
```

Обновите эту строку, изменив `listResult` на `result` и присвоив первую полученную фотографию Mars новой переменной `result`. Присвойте первой фотографии объект с индексом 0.

```
try {
    val result = marsPhotosRepository.getMarsPhotos()[0]
    //...
}
```

В следующей строке обновите параметр, переданный в вызов функции `MarsUiState.Success()`, до строки в следующем коде. Вместо `listResult` используйте данные из нового свойства. Отобразите URL-адрес первого изображения из результатов поиска фотографий.

```
try {
    ...
    MarsUiState.Success("First Mars image URL: ${result.imgSrc}")
}
```

Полный блок попыток теперь выглядит следующим образом:

```
marsUiState = try {
    val result = marsPhotosRepository.getMarsPhotos()[0]
    MarsUiState.Success(
        "    First Mars image URL : ${result.imgSrc}"
    )
}
```

Запустите приложение. Теперь в составном тексте отображается URL первой фотографии Марса. В следующем разделе описано, как заставить приложение отображать изображение в этом URL.



Добавьте композитную функцию `AsyncImage` В этом шаге вы добавите композитную функцию `AsyncImage` для загрузки и отображения одной фотографии Марса. `AsyncImage` - это композитная функция, которая асинхронно выполняет запрос изображения и отображает результат.

```
// Example code, no need to copy over
AsyncImage(
    model = "https://android.com/sample_image.jpg",
    contentDescription = null
)
```


Аргументом модели может быть либо значение `ImageRequest.data`, либо сам `ImageRequest`. В предыдущем примере вы присваиваете значение `ImageRequest.data`, то есть URL-адрес изображения, который равен «`https://android.com/sample_image.jpg`». В следующем примере показано, как присвоить модели сам `ImageRequest`.

```
// Example code, no need to copy over

AsyncImage(
    model = ImageRequest.Builder(LocalContext.current)
        .data("https://example.com/image.jpg")
        .crossfade(true)
        .build(),
    placeholder = painterResource(R.drawable.placeholder),
    contentDescription = stringResource(R.string.description),
    contentScale = ContentScale.Crop,
    modifier = Modifier.clip(CircleShape)
)
```

`AsyncImage` поддерживает те же аргументы, что и стандартный `Image` composable. Кроме того, он поддерживает установку плейсхолдеров/ошибок/обратных вызовов и обратных вызовов `onLoading/onSuccess/onError`. В приведенном примере загружается изображение с обрезкой по кругу и кроссфейдом, а также задается плагин.

`contentDescription` задает текст, используемый службами доступности для описания того, что представляет собой это изображение.

Добавьте в код композитный `AsyncImage`, чтобы отобразить первую полученную фотографию Марса.

В `ui/screens/HomeScreen.kt` добавьте новую композитную функцию `MarsPhotoCard()`, которая принимает `MarsPhoto` и `Modifier`.

```
@Composable
fun MarsPhotoCard(photo: MarsPhoto, modifier: Modifier = Modifier) {
}
```

Внутри композитной функции `MarsPhotoCard()` добавьте функцию `AsyncImage()` следующим образом:

```
import coil.compose.AsyncImage
import coil.request.ImageRequest
import androidx.compose.ui.platform.LocalContext

@Composable
fun MarsPhotoCard(photo: MarsPhoto, modifier: Modifier = Modifier) {
    AsyncImage(
        model = ImageRequest.Builder(context = LocalContext.current)
            .data(photo.imgSrc)
```



```

        .build(),
        contentDescription = stringResource(R.string.mars_photo),
        modifier = Modifier.fillMaxWidth()
    )
}

```

В предыдущем коде вы создаете `ImageRequest`, используя URL-адрес изображения (`photo.imgSrc`), и передаете его в аргумент `model`. Вы используете `contentDescription`, чтобы задать текст для читателей, читающих доступ.

Добавьте `crossfade(true)` в `ImageRequest`, чтобы включить анимацию перекрестного перехода при успешном завершении запроса.

```

@Composable
fun MarsPhotoCard(photo: MarsPhoto, modifier: Modifier = Modifier) {
    AsyncImage(
        model = ImageRequest.Builder(context = LocalContext.current)
            .data(photo.imgSrc)
            .crossfade(true)
            .build(),
        contentDescription = stringResource(R.string.mars_photo),
        modifier = Modifier.fillMaxWidth()
    )
}

```

Обновите составной элемент `HomeScreen`, чтобы он отображал составной элемент `MarsPhotoCard` вместо составного элемента `ResultScreen` при успешном завершении запроса. Ошибка несоответствия типов будет исправлена на следующем шаге.

```

@Composable
fun HomeScreen(
    marsUiState: MarsUiState,
    modifier: Modifier = Modifier
) {
    when (marsUiState) {
        is MarsUiState.Loading -> LoadingScreen(modifier = modifier.fillMaxSize())
        is MarsUiState.Success -> MarsPhotoCard(photo = marsUiState.photos,
        modifier = modifier.fillMaxSize())
        else -> ErrorScreen(modifier = modifier.fillMaxSize())
    }
}

```

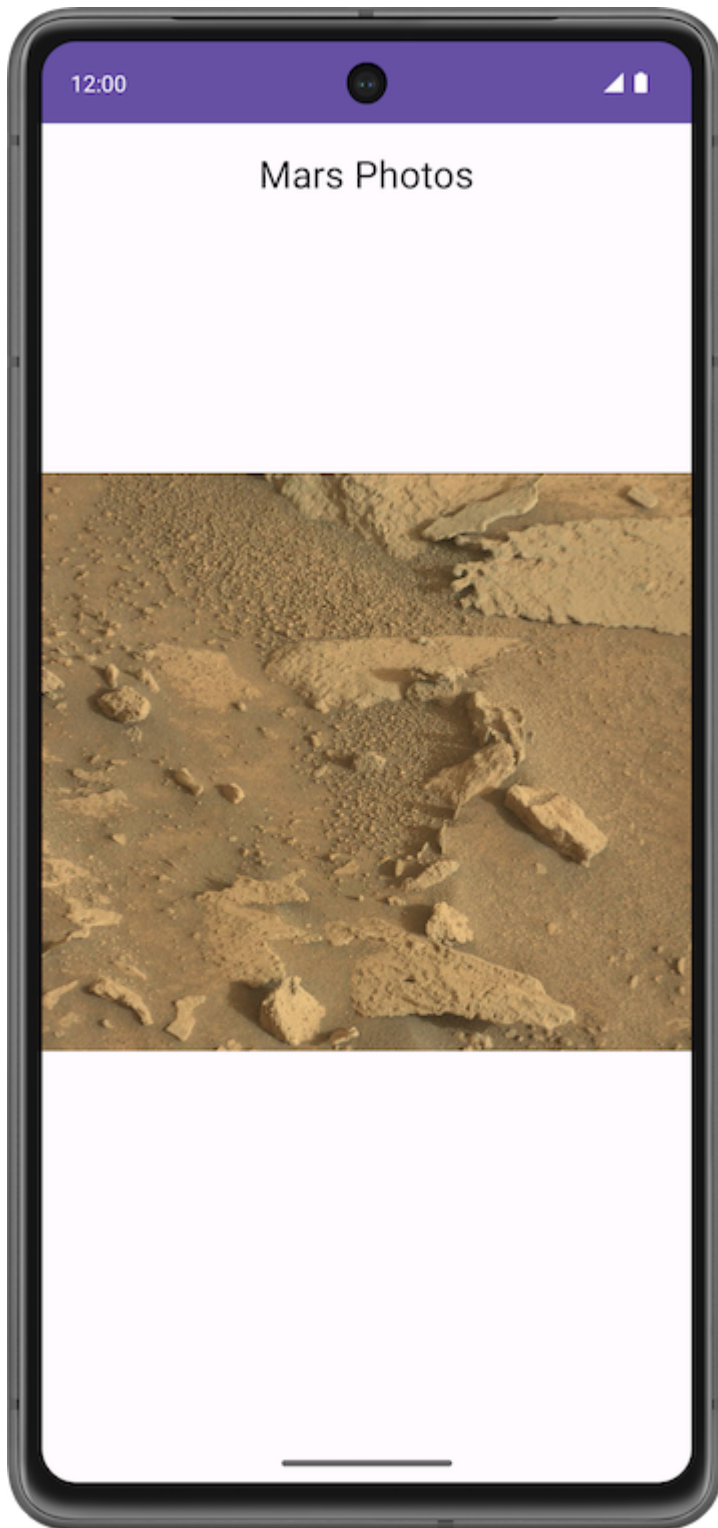
В файле `MarsViewModel.kt` обновите интерфейс `MarsUiState`, чтобы он принимал объект `MarsPhoto` вместо `String`.

```
sealed interface MarsUiState {  
    data class Success(val photos: MarsPhoto) : MarsUiState  
    //...  
}
```

Обновите функцию `getMarsPhotos()`, чтобы передать первый объект фотографии Марса в `MarsUiState.Success()`. Удалите переменную `result`.

```
marsUiState = try {  
    MarsUiState.Success(marsPhotosRepository.getMarsPhotos()[0])  
}
```

Запустите приложение и убедитесь, что оно отображает одно изображение Марса.



Фотография Марса не заполняет весь экран. Чтобы заполнить свободное пространство на экране, в файле `HomeScreen.kt` в `AsyncImage` установите `ContentScale` на `ContentScale.Crop`.

```
import androidx.compose.ui.layout.ContentScale

@Composable
fun MarsPhotoCard(photo: MarsPhoto, modifier: Modifier = Modifier) {
    AsyncImage(
        model = ImageRequest.Builder(context = LocalContext.current)
            .data(photo.imgSrc)
            .crossfade(true)
```

```
        .build(),  
        contentDescription = stringResource(R.string.mars_photo),  
        contentScale = ContentScale.Crop,  
        modifier = modifier,  
    )  
}
```

Запустите приложение и убедитесь, что изображение заполняет экран как по горизонтали, так и по вертикали.

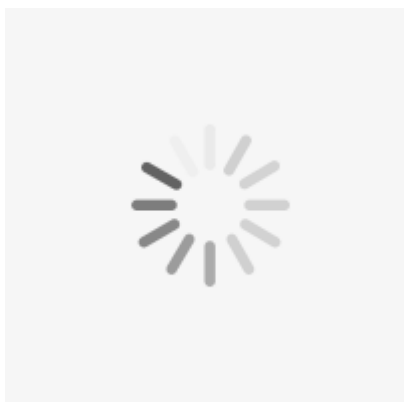


Добавьте изображения загрузки и ошибок. Вы можете улучшить пользовательский опыт в своем приложении, показывая изображение-заполнитель во время загрузки изображения. Вы также можете вывести изображение ошибки, если загрузка не удалась из-за проблемы, например из-за отсутствия или повреждения файла изображения. В этом разделе вы добавите как изображение ошибки, так и изображение-заглушку с помощью `AsyncImage`.

Откройте файл `res/drawable/ic_broken_image.xml` и перейдите на вкладку `Design` или `Split` справа. Для изображения ошибки используйте иконку `broken-image`, которая доступна во встроенной библиотеке иконок. Этот векторный `drawable` использует атрибут `android:tint` для окрашивания значка в серый цвет.



Откройте файл `res/drawable/loading_img.xml`. Этот `drawable` представляет собой анимацию, которая вращает изображение `drawable, loading_img.xml`, вокруг центральной точки. (В предварительном просмотре анимация не видна).



Вернитесь к файлу `HomeScreen.kt`. В составной `MarsPhotoCard` обновите вызов `AsyncImage()`, чтобы добавить атрибуты `error` и `placeholder`, как показано в следующем коде:

```
import androidx.compose.ui.res.painterResource
```



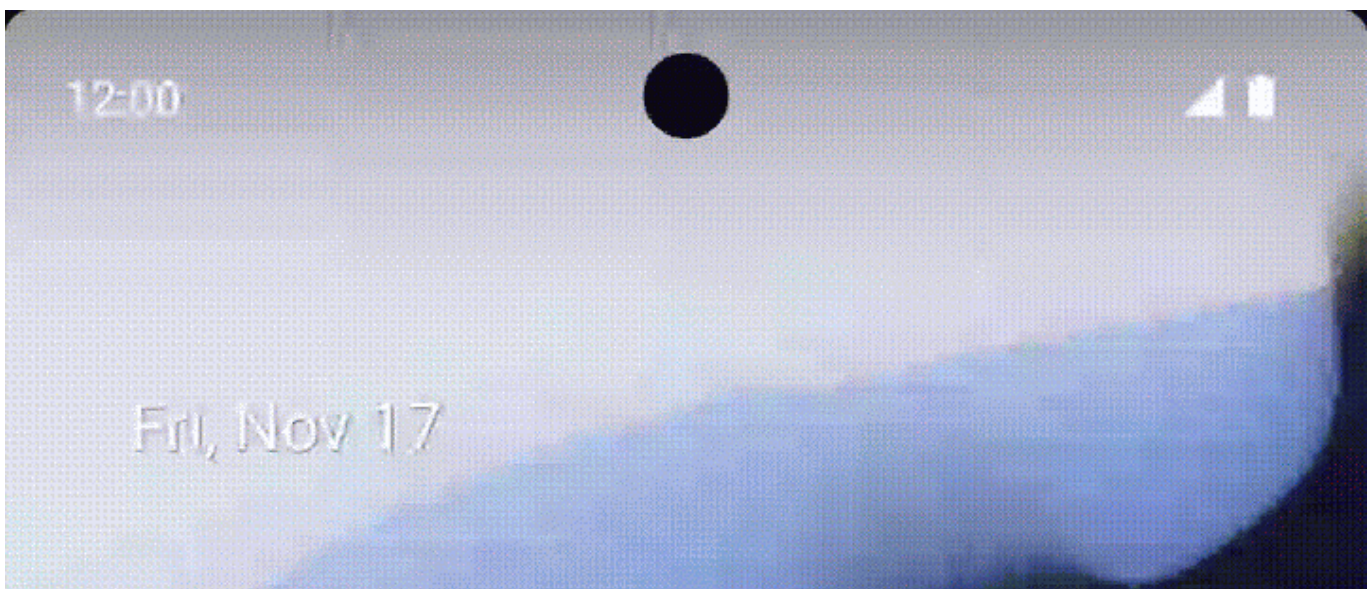
```
@Composable
fun MarsPhotoCard(photo: MarsPhoto, modifier: Modifier = Modifier) {
    AsyncImage(
        // ...
        error = painterResource(R.drawable.ic_broken_image),
        placeholder = painterResource(R.drawable.loading_img),
        // ...
    )
}
```

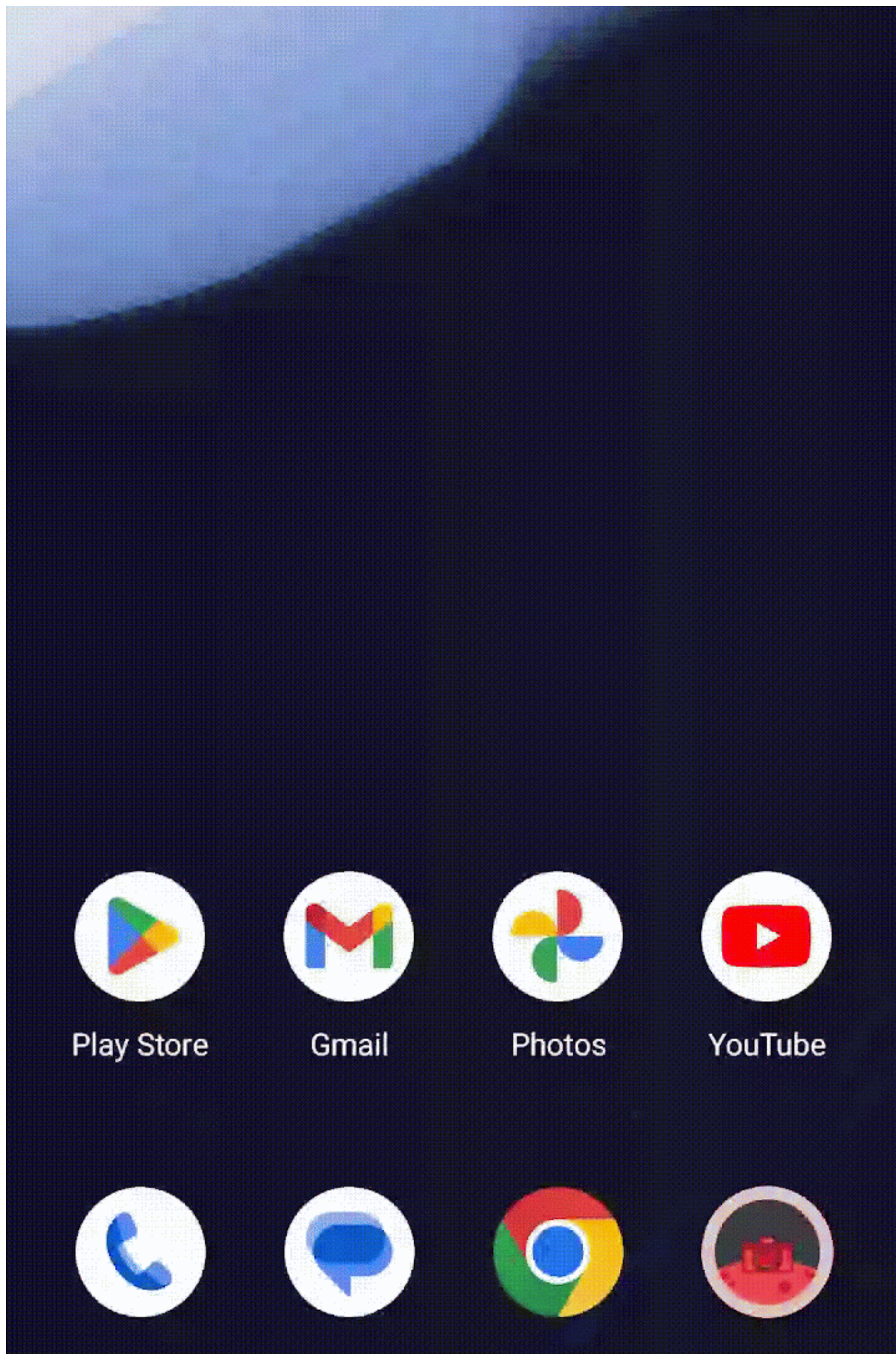
Этот код устанавливает загрузочное изображение-заполнитель, которое будет использоваться при загрузке (drawable loading_img). Он также устанавливает изображение, которое будет использоваться в случае неудачной загрузки изображения (drawable ic_broken_image).

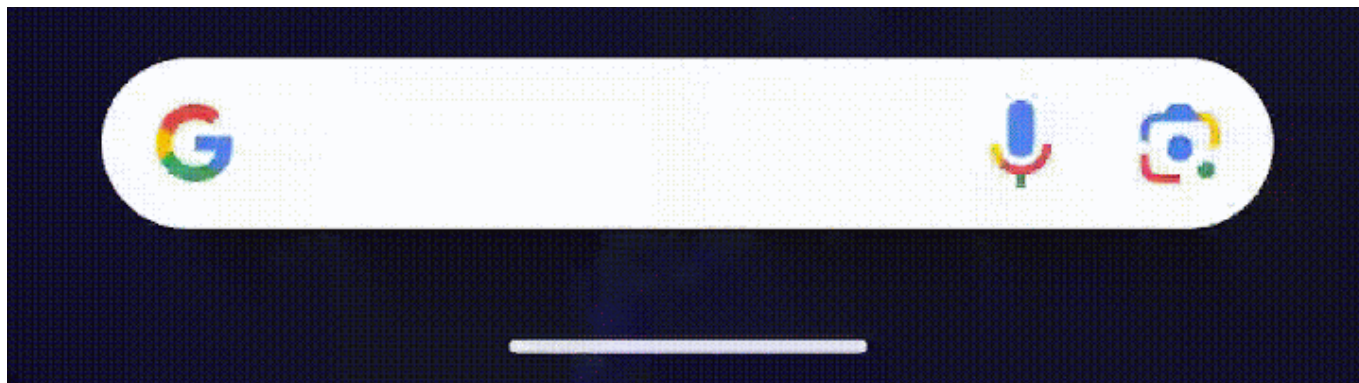
Полный состав MarsPhotoCard теперь выглядит следующим образом:

```
@Composable
fun MarsPhotoCard(photo: MarsPhoto, modifier: Modifier = Modifier) {
    AsyncImage(
        model = ImageRequest.Builder(context = LocalContext.current)
            .data(photo.imgSrc)
            .crossfade(true)
            .build(),
        error = painterResource(R.drawable.ic_broken_image),
        placeholder = painterResource(R.drawable.loading_img),
        contentDescription = stringResource(R.string.mars_photo),
        contentScale = ContentScale.Crop
    )
}
```

Запустите приложение. В зависимости от скорости вашего сетевого подключения вы можете ненадолго увидеть загрузочное изображение, пока Coil загружает и отображает изображение свойств. Но вы еще не увидите значок сломанного изображения, даже если отключите сеть - это можно исправить в последнем задании коделаба.







```
{style="width:400px"}
```

4. Отображение сетки изображений с помощью LazyVerticalGrid

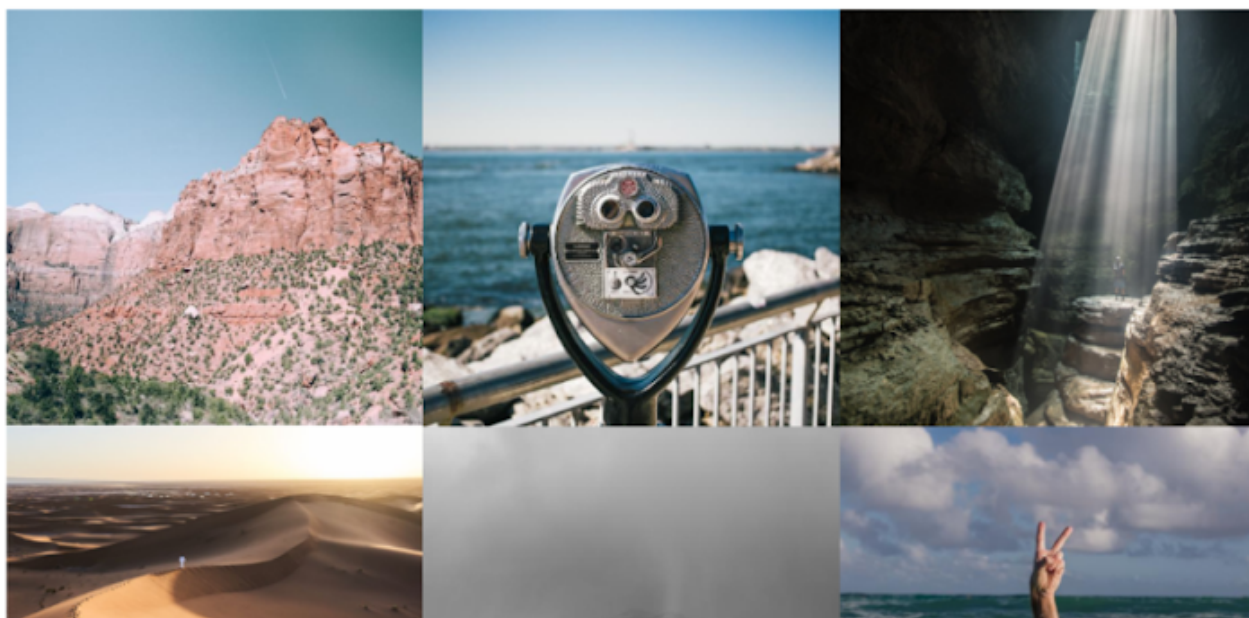
Теперь ваше приложение загружает фотографию Марса из интернета, первый элемент списка MarsPhoto. Вы использовали URL-адрес изображения из данных фотографии Марса, чтобы заполнить AsyncImage. Однако цель состоит в том, чтобы ваше приложение отображало сетку изображений. В этой задаче вы используете LazyVerticalGrid с менеджером компоновки Grid для отображения сетки изображений.

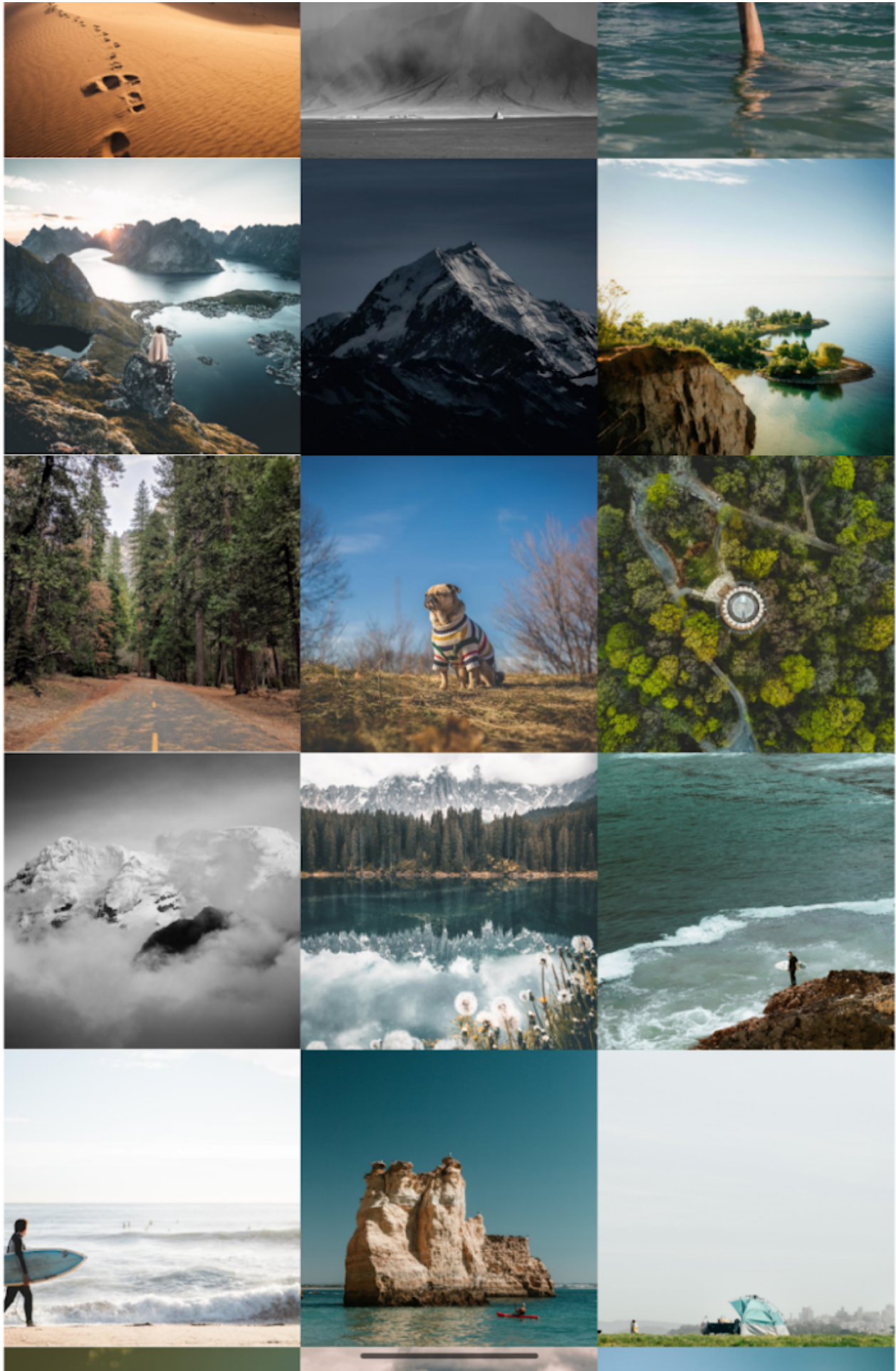
Ленивые сетки Композиты LazyVerticalGrid и LazyHorizontalGrid обеспечивают поддержку отображения элементов в сетке. Ленивая вертикальная сетка отображает элементы в вертикально прокручиваемом контейнере, расположенном в нескольких столбцах, а ленивая горизонтальная сетка ведет себя аналогичным образом по горизонтальной оси.

11:00



Grid





С точки зрения дизайна, Grid Layout лучше всего подходит для отображения фотографий Марса в виде иконок или изображений.

Параметр `columns` в `LazyVerticalGrid` и параметр `rows` в `LazyHorizontalGrid` управляют тем, как ячейки формируются в столбцы или строки. Следующий пример кода отображает элементы в сетке, используя `GridCells.Adaptive` для установки ширины каждого столбца не менее 128.dp:

```
// Sample code - No need to copy over

@Composable
fun PhotoGrid(photos: List<Photo>) {
    LazyVerticalGrid(
        columns = GridCells.Adaptive(minSize = 150.dp)
    ) {
        items(photos) { photo ->
            PhotoItem(photo)
        }
    }
}
```

`LazyVerticalGrid` позволяет задать ширину элементов, после чего сетка вмещает столько столбцов, сколько возможно. После подсчета количества столбцов сетка распределяет оставшуюся ширину поровну между столбцами. Этот адаптивный способ изменения размера особенно полезен для отображения наборов элементов на экранах разных размеров.

Примечание: Если вы знаете точное количество столбцов, которые необходимо использовать, вы можете вместо этого предоставить экземпляр `GridCells.Fixed`, содержащий необходимое количество столбцов.

В этом кодовом примере для отображения фотографий Марса используется `LazyVerticalGrid`, компонуемый с `GridCells.Adaptive`, с шириной каждого столбца 150.dp.

Клавиши элементов Когда пользователь прокручивает сетку (`LazyRow` внутри `LazyColumn`), позиция элемента списка меняется. Однако из-за смены ориентации или при добавлении или удалении элементов пользователь может потерять позицию прокрутки в строке. Ключи элементов помогают сохранять позицию прокрутки в зависимости от ключа.

Предоставляя ключи, вы помогаете Compose правильно обрабатывать переупорядочивание. Например, если элемент содержит запомненное состояние, установка ключей позволяет Compose перемещать это состояние вместе с элементом при изменении его позиции.

Добавить `LazyVerticalGrid` Добавьте композит для отображения списка фотографий Марса в виде вертикальной сетки.

В файле `HomeScreen.kt` создайте новую композитную функцию `PhotosGridScreen()`, которая принимает в качестве аргументов список `MarsPhoto` и модификатор.

```
@Composable
fun PhotosGridScreen(
    photos: List<MarsPhoto>,
    modifier: Modifier = Modifier,
    contentPadding: PaddingValues = PaddingValues(0.dp),
```

```
) {  
}
```

Внутри композита PhotosGridScreen добавьте LazyVerticalGrid со следующими параметрами.

```
import androidx.compose.foundation.lazy.grid.LazyVerticalGrid  
import androidx.compose.foundation.lazy.grid.GridCells  
import androidx.compose.foundation.layout.fillMaxWidth  
import androidx.compose.foundation.layout.PaddingValues  
import androidx.compose.ui.unit.dp  
  
@Composable  
fun PhotosGridScreen(  
    photos: List<MarsPhoto>,  
    modifier: Modifier = Modifier,  
    contentPadding: PaddingValues = PaddingValues(0.dp),  
) {  
    LazyVerticalGrid(  
        columns = GridCells.Adaptive(150.dp),  
        modifier = modifier.padding(horizontal = 4.dp),  
        contentPadding = contentPadding,  
    ) {  
    }  
}
```

Чтобы добавить список элементов, внутри лямбды LazyVerticalGrid вызовите функцию items(), передав в нее список MarsPhoto и ключ элемента в виде photo.id.

```
import androidx.compose.foundation.lazy.grid.items  
  
@Composable  
fun PhotosGridScreen(  
    photos: List<MarsPhoto>,  
    modifier: Modifier = Modifier,  
    contentPadding: PaddingValues = PaddingValues(0.dp),  
) {  
    LazyVerticalGrid(  
        // ...  
    ) {  
        items(items = photos, key = { photo -> photo.id }) {  
        }  
    }  
}
```

Чтобы добавить содержимое, отображаемое одним элементом списка, определите лямбда-выражение items. Вызовите MarsPhotoCard, передав фотографию.


```
items(items = photos, key = { photo -> photo.id }) {  
    photo -> MarsPhotoCard(photo)  
}
```

Обновите составной элемент HomeScreen, чтобы он отображал составной элемент PhotosGridScreen вместо составного элемента MarsPhotoCard при успешном завершении запроса.

```
when (marsUiState) {  
    // ...  
    is MarsUiState.Success -> PhotosGridScreen(marsUiState.photos, modifier)  
    // ...  
}
```

В файле MarsViewModel.kt обновите интерфейс MarsUiState, чтобы он принимал список объектов MarsPhoto вместо одного MarsPhoto. Составной экран PhotosGridScreen принимает список объектов MarsPhoto.

```
sealed interface MarsUiState {  
    data class Success(val photos: List<MarsPhoto>) : MarsUiState  
    //...  
}
```

В файле MarsViewModel.kt обновите функцию getMarsPhotos(), чтобы передать список объектов фотографий Марса в MarsUiState.Success().

```
marsUiState = try {  
    MarsUiState.Success(marsPhotosRepository.getMarsPhotos())  
}
```

Запустите приложение.



{style="width:400px"}

Обратите внимание, что вокруг каждой фотографии нет подложки, а соотношение сторон у разных фотографий разное. Чтобы исправить эти проблемы, можно добавить композитную карту.

Добавьте составную карту В файле HomeScreen.kt, в композите MarsPhotoCard, добавьте Card с высотой 8.dp вокруг AsyncImage. Присвойте аргумент модификатора композиту Card.

```
import androidx.compose.material.Card
import androidx.compose.foundation.layout.aspectRatio
import androidx.compose.foundation.layout.padding

@Composable
```

```
fun MarsPhotoCard(photo: MarsPhoto, modifier: Modifier = Modifier) {

    Card(
        modifier = modifier,
        elevation = CardDefaults.cardElevation(defaultElevation = 8.dp)
    ) {

        AsyncImage(
            model = ImageRequest.Builder(context = LocalContext.current)
                .data(photo.imgSrc)
                .crossfade(true)
                .build(),
            error = painterResource(R.drawable.ic_broken_image),
            placeholder = painterResource(R.drawable.loading_img),
            contentDescription = stringResource(R.string.mars_photo),
            contentScale = ContentScale.Crop,
            modifier = Modifier.fillMaxWidth()
        )
    }
}
```

Чтобы исправить соотношение сторон, в PhotosGridScreen() обновите модификатор для MarsPhotoCard().

```
@Composable
fun PhotosGridScreen(photos: List<MarsPhoto>, modifier: Modifier = Modifier) {
    LazyVerticalGrid(
        //...
    ) {
        items(items = photos, key = { photo -> photo.id }) { photo ->
            MarsPhotoCard(
                photo,
                modifier = modifier
                    .padding(4.dp)
                    .fillMaxWidth()
                    .aspectRatio(1.5f)
            )
        }
    }
}
```

Обновите предварительный просмотр экрана результатов для предварительного просмотра PhotosGridScreen(). Подражайте данным с пустыми URL-адресами изображений.

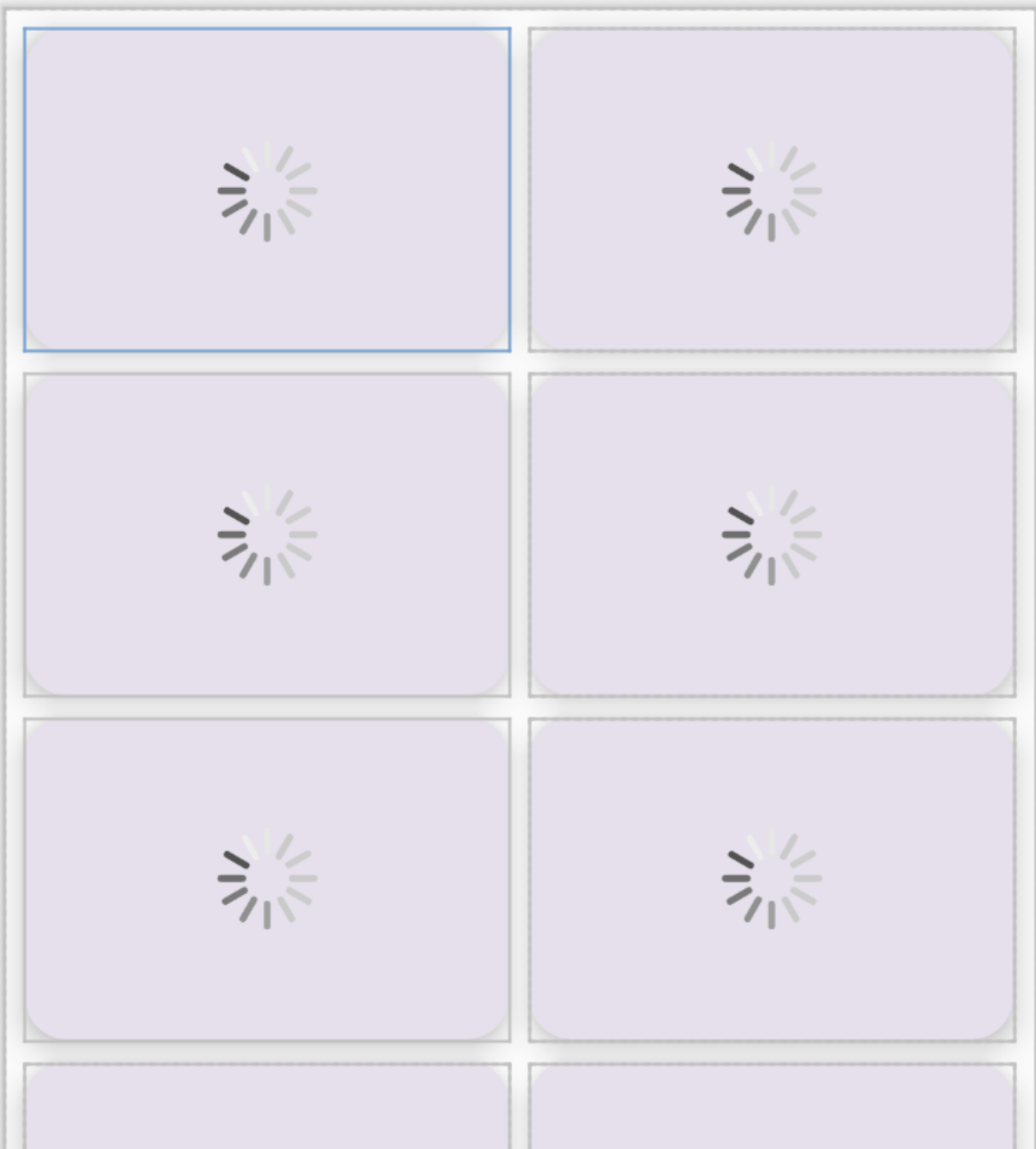
```
@Preview(showBackground = true)
@Composable
fun PhotosGridScreenPreview() {
    MarsPhotosTheme {
        val mockData = List(10) { MarsPhoto("$it", "") }
    }
}
```

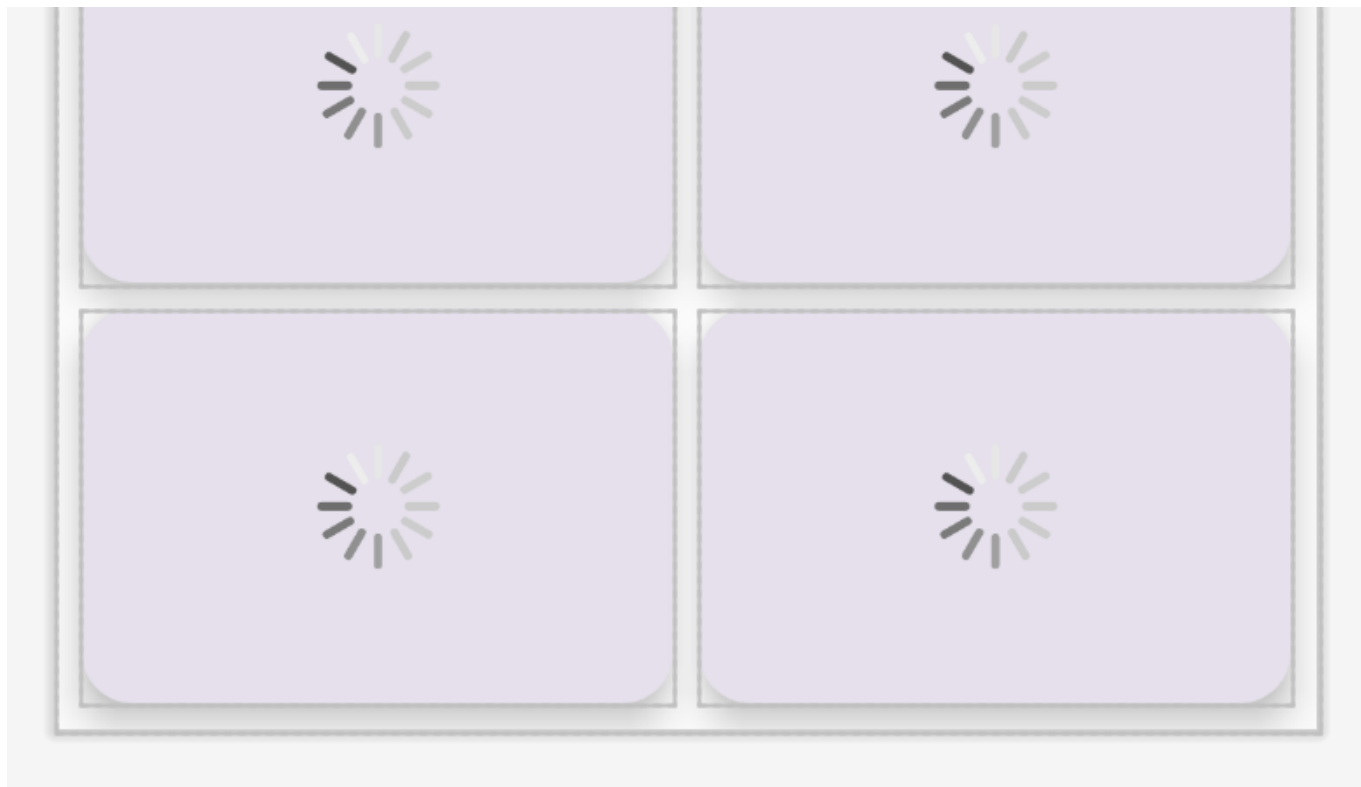
```
        PhotosGridScreen(mockData)
    }
}
```

Макетные данные: Некоторые функции могут зависеть от других объектов или данных. Чтобы изолировать поведение этих функций, необходимо заменить другие объекты или данные на имитационные или поддельные объекты и данные, чтобы имитировать поведение реального объекта. Макетирование - это процесс создания объектов, которые имитируют поведение реальных объектов. В основном имитация используется в модульном тестировании.

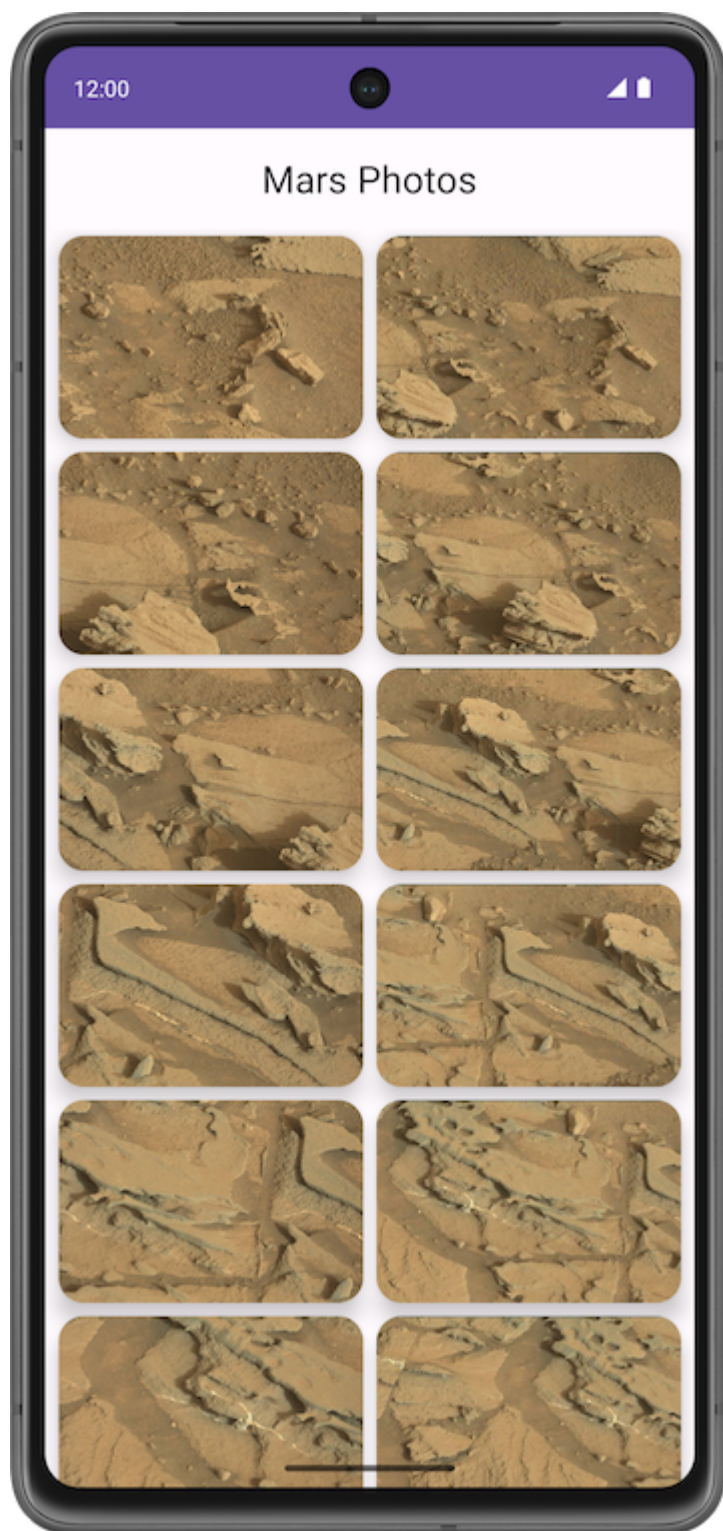
Поскольку имитационные данные имеют пустые URL-адреса, в предварительном просмотре фотосетки отображается загрузка изображений.

PhotosGridScreenPreview





Запустите приложение.



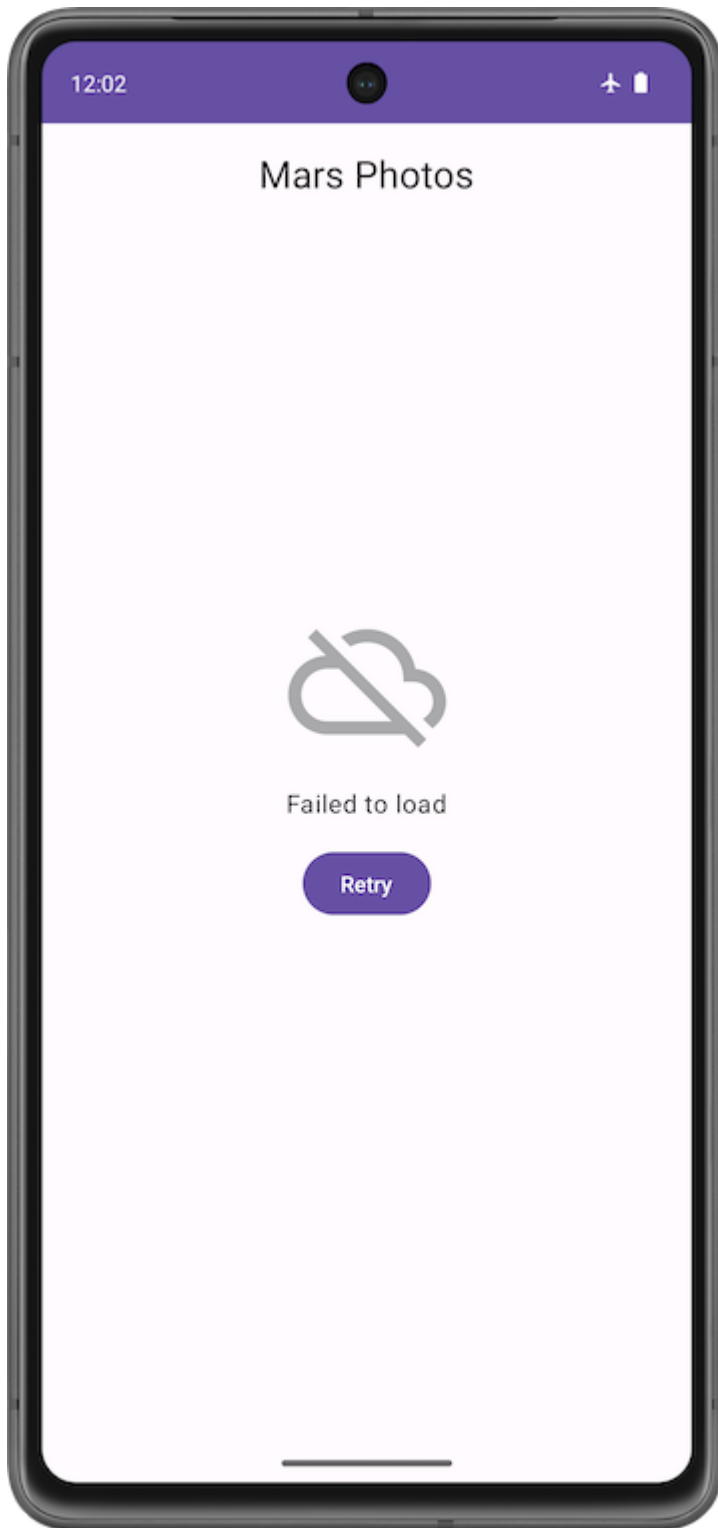
Во время работы приложения включите режим самолета. Прокрутите изображения в эмуляторе. Изображения, которые еще не загрузились, отображаются в виде иконок с неработающими изображениями. Это изображение, которое вы передали библиотеке изображений Coil для отображения в случае сетевой ошибки или невозможности получения изображения.



Отличная работа! Вы смоделировали ошибку подключения к сети, включив режим самолета в эмуляторе или устройстве.

5. Добавьте действие повтора

В этом разделе вы добавите кнопку повторного действия и получите фотографии при нажатии на кнопку.



Добавьте кнопку на экран ошибок. В файле `HomeScreen.kt` обновите составной элемент `ErrorScreen()`, чтобы включить в него лямбда-параметр `retryAction` и кнопку.

```
@Composable
fun ErrorScreen(retryAction: () -> Unit, modifier: Modifier = Modifier) {
    Column(
        // ...
    ) {
        Image(
            // ...
        )
    }
}
```

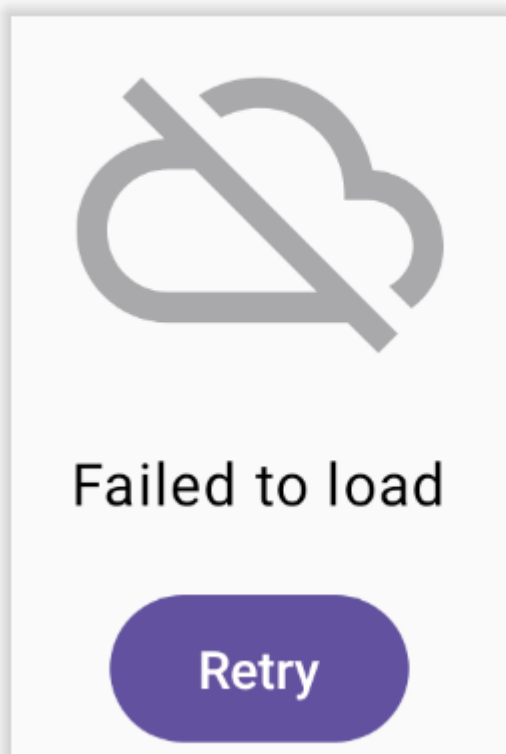
```
Text(//...)
Button(onClick = retryAction) {
    Text(stringResource(R.string.retry))
}
}
```

Примечание: Убедитесь, что `ErrorScreenPreview()` соответствует следующему фрагменту кода:

```
@Preview(showBackground = true)
@Composable
fun ErrorScreenPreview() {
    MarsPhotosTheme {
        ErrorScreen({})
    }
}
```

Проверьте предварительный просмотр

ErrorScreenPreview



Обновите композит `HomeScreen()`, чтобы передать лямбду `retry`.


```

@Composable
fun HomeScreen(
    marsUiState: MarsUiState, retryAction: () -> Unit, modifier: Modifier =
    Modifier
) {
    when (marsUiState) {
        //...

        is MarsUiState.Error -> ErrorScreen(retryAction, modifier =
        modifier.fillMaxSize())
    }
}

```

В файле `ui/theme/MarsPhotosApp.kt` обновите вызов функции `HomeScreen()`, чтобы установить лямбда-параметр `retryAction` на `marsViewModel::getMarsPhotos`. Это позволит получить марсианские фотографии с сервера.

```

HomeScreen(
    marsUiState = marsViewModel.marsUiState,
    retryAction = marsViewModel::getMarsPhotos
)

```

6. Обновление теста ViewModel

`MarsUiState` и `MarsViewModel` теперь могут работать со списком фотографий вместо одной фотографии. В текущем состоянии `MarsViewModelTest` ожидает, что класс данных `MarsUiState.Success` будет содержать свойство `string`. Поэтому тест не компилируется. Вам нужно обновить тест `marsViewModel_getMarsPhotos_verifyMarsUiStateSuccess()`, чтобы он утверждал, что `MarsViewModel.marsUiState` равен состоянию `Success`, которое содержит список фотографий.

Откройте файл `rules/MarsViewModelTest.kt`. В тесте `marsViewModel_getMarsPhotos_verifyMarsUiStateSuccess()` измените вызов функции `assertEquals()`, чтобы сравнить состояние `Success` (передача поддельного списка фотографий в параметр `photos`) с состоянием `marsViewModel.marsUiState`.

```

@Test
fun marsViewModel_getMarsPhotos_verifyMarsUiStateSuccess() =
    runTest {
        val marsViewModel = MarsViewModel(
            marsPhotosRepository = FakeNetworkMarsPhotosRepository()
        )
        assertEquals(
            MarsUiState.Success(FakeDataSource.photosList),
            marsViewModel.marsUiState
        )
    }

```



Теперь тест компилируется, запускается и проходит!

Заключение

Поздравляем вас с созданием приложения **Mars Photos**! Пришло время продемонстрировать свое приложение с реальными фотографиями Марса своей семье и друзьям.